

El lenguaje ensamblador del PIC16F84A

Introducción

El juego de instrucciones de un microprocesador o microcontrolador es el conjunto de entradas binarias que producen acciones definidas durante un ciclo de instrucción. Un juego de instrucciones es para el microcontrolador lo mismo que una tabla de verdad es para una puerta lógica, un registro de desplazamiento o un sumador. Por supuesto, las acciones que realiza un microcontrolador con cada instrucción, son más complejas que las que realizan los dispositivos y puertas antes mencionados.

Una instrucción es un patrón de dígitos binarios el cual debe estar a disposición del microcontrolador en el tiempo y forma que éste lo requiera.

Por ejemplo, cuando el procesador de un microcontrolador PIC16F84A recibe el patrón binario de 12 bits '0000 0100 0000' en el momento adecuado, significa: Clear (borrar o poner a cero) el registro W, y corresponde a la instrucción CLRW.

En instrucciones del PIC16F84A se muestra el juego de instrucciones completo del PIC16F84A

Para los PIC se han creado unas instrucciones y una estructura de programa que los hacen más sencillos y atractivos todavía..

Las instrucciones de los microcontrolador PIC cumplen con las siguientes condiciones:

- Juego de instrucciones reducido: Por ejemplo, solo existen 35 instrucciones en el PIC16F84A.
- Sencillas y rápidas: La mayoría se ejecuta en un ciclo de instrucción, y solo las de salto precisan 2 ciclos. El ciclo de instrucción consta de 4 ciclos de reloj principal. De esta manera un dispositivo con un cristal de cuarzo de 20 MHz realiza 5 millones de instrucciones por segundo.
- Ortogonalidad: La ubicación de los operandos que manejan es muy flexible. Cualquier objeto del procesador puede actuar como origen o como destino.
- Formato uniforme de las instrucciones: Todas las instrucciones tienen una longitud fija de bits. Esta característica significa un notable ahorro de la memoria de programa y una facilidad en la construcción de compiladores.
- Formato uniforme de los datos.

Un programa es una serie de instrucciones mediante las cuales un sistema basado en CPU realiza una tarea en particular y la forma mas simple de realizar un programa es mediante el lenguaje ensamblador (Ver Sistemas microprogramables, Lenguajes de programación para sistemas basados en CPU).

Podríamos decir que el lenguaje ensamblador "*es complejo por su sencillez*". Esto quiere decir que, a diferencia de los lenguajes de alto nivel, aquí no hay funciones que solucionen y simplifiquen algoritmos, si no que hay que implementar hasta los pasos más elementales.

Programa en ensamblador

Para introducir un programa en ensamblador en un sistema basado en CPU tenemos que traducirlo a hexadecimal o a binario. Para realizarlo a su vez se utiliza un programa de ordenador, llamado programa ensamblador. Éste sería un ejemplo de programación en ensamblador o mnemónicos:

```
start    org      0
         movlw    0Eh
         movwf    REG1
         movlw    100
         addwf    REG1,1
         end
```

Desventajas del ensamblador

Existe una gran diferencia entre el juego de instrucciones de un sistema basado en CPU y las tareas que este debe realizar. Las instrucciones tienden a hacer cosas como: sumar contenidos de dos registros, desplazar el contenido de un acumulador un bit, o colocar un nuevo valor en el contador de programa.

Por otro lado, centrandonos en el caso de los microcontroladores, este deberá hacer cosas como: reaccionar cuando una entrada digital se activa, comprobar si un valor analógico se ha excedido de un cierto

umbral, activar un relé en un momento determinado, mostrar resultados en un panel LCD, comunicarse vía serie con otros dispositivos, etc. El programador en lenguaje ensamblador debe "traducir" estas tareas a secuencias de simples instrucciones. Esto no suele ser fácil y consume tiempo de trabajo.

Otro inconveniente es la no portabilidad. Cada microprocesador o microcontrolador posee su propio juego de instrucciones en el y su propia arquitectura interna.

Un programa en ensamblador escrito para el PIC16F84A, no correrá en un 65C02, Z80, 8080, 8051, o cualquier otro sistema basado en CPU. Incluso dentro de los PIC hay diferencias entre las distintas gamas como número y tipo de instrucciones, recursos disponibles, dirección de registros o uso de la memoria.

Para solucionar estos problemas están los programas de alto nivel, como el lenguaje C o Basic.

Mnemónicos

La tarea principal del ensamblador es la traducción de los códigos de operación en mnemónico en sus equivalentes binarios.

El ensamblador realiza ésta tarea usando una tabla como si lo hiciésemos "a mano" pero además debe determinar cuantos operandos requiere la instrucción y de que tipo. Esto es un poco complejo; algunas instrucciones (como CLRW, SLEEP) no tienen operandos, otras (ADDLW 13, GOTO FIN) tienen una, mientras que otras (BSF STATUS,C o BTFSS PORTA,O) requieren dos.

Identificación de elementos

Flags o Banderas

Los Flags o banderas son marcadores, representados por bits dentro del registro de **STATUS**, los mas importantes son:

- **Z:** Flag de cero, se pone a 1 cuando una operación que le afecta da como resultado un 0.
- **C:** Flag de Carry, se pone a 1 cuando la operación que le afecta sobrepasa el nivel de representación del procesador, en nuestro caso es 8 bits, de esta manera si sumamos a **1111 1111 b** un **0000 0011 b** el resultado sería **0000 0010 b** y el **bit de Carry** pasaría a 1.
- **DC:** Flag de carry del nibbles inferior, este se comporta igual que el **bit de Carry**, solo que el limite de representación son los 4 bits inferiores, de esta manera si tenemos **0000 1111 b** y sumamos **0000 0111 b**, el resultado será **0001 0110 b** y el **bit de DC** se pone a 1, el **bit de Carry** estará a 0 al no superarse los 8 bits y el **bit Z** a 0 al ser el número diferente de 0.

Registros

Un registro es un espacio en la memoria de datos del microcontrolador en el que podemos guardar información, existen también unos registros en los cuales podemos configurar el microcontrolador o saber el estado de este o algunos de sus periféricos.

Un registro está compuesto por 8 bits los cuales se representan dándoles un numero según su posición, de esta manera el bit menos significativo (LSB) se le da el número 0 y el más significativo (MSB) el 7.

BIT	7	6	5	4	3	2	1	0
REGISTRO	X	X	X	X	X	X	X	X

Donde X puede ser 1 ó 0.

A los bits del 0 al 3 se les denomina nibbles inferior, y del 4 al 7 se denominan nibbles superior.

La forma de representación de parte de los bits de un registro suele ser:

Registro<3:0>

lo que indica los bits del 3 al 0 del registro.

De esta forma, para identificar el BIT Z de STATUS se pondría:

STATUS<2>

Elementos de una instrucción

En el caso del PIC16F84A y los de los PIC de la gama media cada instrucción está formada por una palabra de 14 bits que utiliza un tipo de código denominado **OPCODE** (Código de Operación), que especifica el **mnemónico** de la operación y los operandos que correspondan, que son los datos con los que opera la instrucción.

Ejemplo, instrucción *CLRF f*.

CLRF				CLRF
Clear f				
Operación	00 h → f 1 → Z			
Sintaxis	[Etiqueta] CLRF f			
Operadores	0 < f < 127			
Ciclos	1			
OPCODE	00	0001	1fff	Ffff
Descripción	Se borra el contenido del registro f y el flag Z se activa			

El OPCODE de *CLRF f* es en binario "0000011fffff" donde "fffff" se sustituiría por el registro que se quiera borrar. **f** es una de las abreviaturas que se utilizan para describir las instrucciones del PIC usados en el lenguaje ensamblador y que son:

- **f** Representa un registro cualquiera de la memoria de datos.
- **w** Registro de trabajo (Working Register).
- **b** Dirección de un bit dentro de un registro de 8 bits (0-7).
- **l** ó **k** Literal o constante de 8 bits.
- **d** Bit de destino, 0 ó 1.
- **x** Los bits que estén representados por este tipo de dato no tienen ninguna función y su valor lo define el compilador.

A continuación se explican con más detalle:

f (file register)

Este carácter se usa para definir registros de cualquier tipo. Cualquier instrucción que contenga este campo, contendrá la dirección de un registro, no su contenido. Un registro puede variar entre las direcciones 00h y 7Fh.

En el caso de los registros especiales en vez de la dirección podemos poner directamente el nombre del registro que el ensamblador se encargará luego de traducir a la dirección real.

Ejemplo, instrucción *BSF f,b*, Pone a 1 el bit **b** del registro **f**.

En lugar de poner:

BSF 03,5

podemos poner:

BSF STATUS,5

con lo se pone a 1 el bit 5 del registro STATUS.

w (working register)

w da nombre al acumulador de los PICs, el cual lo vimos anteriormente cuando tratamos los registros. Este no es un registro situado en un banco de memoria, si no que es independiente. A diferencia que el anterior, cuando nos referimos a él, nos referimos al contenido. Su uso es muy sencillo, pues lo usaremos principalmente para pasar información de un registro a otro, o para contener la información entre dos o más instrucciones.

b (bit address dentro de un registro)

Esta letra define la dirección de un bit dentro de un byte. En ciertas ocasiones en vez de modificar o acceder a bytes tendremos que modificar o acceder a bits. De esta manera podemos especificar a una instrucción que posición ocupa el bit sobre el cual recaerá la acción que esta ejecute. Al igual que en los registros especiales, podemos poner directamente el nombre de un bit dentro de un registro.

Ejemplo:

En lugar de:

BSF STATUS,5

ponemos:

BSF STATUS,RP0

l o k (literal)

Este valor será almacenado en la propia instrucción en tiempo de ensamblado, esto significa que son los valores que introducimos en las instrucciones para que trabaje con ellos (independientemente de los datos que podamos almacenar o contener en la EEPROM de datos). El valor que podemos introducir dentro de un literal está comprendido entre 0 y 255, ya que es el máximo que puede representar un byte.

d (destiny bit)

Donde encontremos esta letra, debemos especificar donde se almacenará el resultado de una instrucción, en **w** o en un registro. Puesto que esto no es un lenguaje de alto nivel, no podemos almacenar el resultado de una operación sobre una tercera variable o registro, así que este deberá ser almacenado en el registro origen (sobrescribiéndose), o en el acumulador. Esto se define a través de dos valores:

- 1: El resultado se almacenará en **f**.
 - 0: El resultado se almacenará en **w**.
-

Estructura de un programa en ensamblador

Para hacer la tarea del programador más grata, se usan algunas convenciones. Cada uno puede adoptar las que más le agraden y ayuden para ser más productivo. En general, las convenciones son cualquier acción que facilita la revisión y comprensión de un programa, especialmente el que uno mismo ha escrito cuando tiene que revisarlo algunos meses después. Comentamos algunas convenciones que usaremos:

- Los ficheros de código fuente llevarán la extensión *.ASM
- Los ficheros de listado llevarán la extensión *.LST
- Los ficheros de código objeto llevarán la extensión *.OBJ
- Los ficheros de errores llevarán la extensión *.ERR
- Los ficheros ejecutables en formato Intel Hex llevarán la extensión *.HEX
- Comentario descriptivo del programa (utilizar una cabecera estandarizada).
- Definir el microcontrolador que se usará (con las directivas LIST e INCLUDE).
- Introducir las opciones de compilación (que serán vistas más adelante) (opcional).
- Establecer las constantes que se usarán (con la directiva EQU).
- Reservar espacios de memoria (directiva RES) (si es necesario).
- Configurar los puertos.
- Desarrollar el programa con comentarios, en lo posible explicando cada línea de código..
- Los mnemónicos escritos en minúscula y las constantes y variables en mayúscula hacen que el código escrito sea más visible.
- Colocar las rutinas en el mismo sitio, todas contiguas.
- Dibujar diagramas de flujo o escribir pseudocódigo.

Su estructura en un programa ejemplo muy simple:

```
*****
;Programa E001.asm                      Fecha: 3 Diciembre 2004
;Este programa suma dos valores inmediatos (7+8) y el resultado
;lo deposita en la posición 0x10
;Revisión: 0.0                          Programa para PIC16F84
;Velocidad de reloj: 4 MHz               Instrucción: 1Mz=1 us
;Perro Guardián: deshabilitado          Tipo de Reloj: XT
;Protección de código: OFF
*****
LIST      p=16F84 ;Tipo de PIC
*****
RESULTADO EQU 0x10 ;Define la posición del resultado
*****
ORG 0      ;Comando que indica al Ensamblador
           ;la dirección de la memoria de programa
           ;donde situar la siguiente instrucción
*****
INICIO     movlw   0x07 ;Carga primer sumando en W
           addlw   0x08 ;Suma W con segundo sumando
           movwf   RESULTADO ;Almacena el resultado

           END        ;Fin del programa fuente
*****
```

Cabecera

Tipo de microcontrolador

Definición de constantes

Comentarios

Programa

Hemos visto la estructura general. Ahora veremos la posición de los elementos del código por 4 columnas:

Etiquetas Operación Operandos Comentarios

Etiquetas	Operación	Operandos	Comentarios
INICIO	movlw	0x07	;Carga primer sumando en W
	addlw	0x08	;Suma W con segundo sumando
	movwf	RESULTADO	;Almacena el resultado
	END		;Fin del programa fuente

- **Columna 1: Etiquetas.** Las etiquetas se rigen por las siguientes normas:
 - Debe situarse en la primera columna.
 - Debe contener únicamente caracteres alfanuméricos.
 - El máximo de caracteres es de 31.

- **Columna 2: Operación.** En esta columna se situarán las instrucciones. El campo del código de operación es el único que nunca puede estar vacío; éste siempre contiene una instrucción o una directiva del ensamblador.
- **Columna 3: Operandos** El campo de operandos o de dirección puede contener una dirección o un dato, o puede estar en blanco. Normalmente contendrá registros o literales con los que se operará (f, l o k, b y w).
- **Columna 4: Comentario.** El campo del comentario o de etiquetas es opcional. Aquí se situará cualquier comentario personalizado que deseemos. Estos son útiles para saber qué hace un programa sin tener que descifrar el código entero. El compilador (ensamblador) ignorará todo texto más allá del carácter punto y coma ";".

Los comentarios generalmente se sitúan en la cuarta columna para describir la acción de una línea de código, pero pueden situarse en cualquier parte de programa para describir cualquier otro evento, siempre que estén después del carácter ";" (semicolon en inglés).

Normalmente las columnas son separadas por una tabulación. El espacio mínimo entre dos columnas es de un carácter, que puede ser un espacio en vez de una tabulación.

Delimitadores (separación entre campos)

- Los campos van separados sólo con espacios y/o tabulaciones. No agregue nunca otros caracteres (comas, puntos, etc.)
- No utilice espacios extra, particularmente después de comas que separan operandos. (Ej: movlw 5, w)
- No use caracteres delimitadores (espacios y tabulaciones) en nombres o etiquetas.

Etiquetas (label)

Las etiquetas se sitúan a la izquierda de las instrucciones y sirven para agrupar fragmentos de código. Estos fragmentos pueden ser de dos tipos:

- El primer tipo no es un fragmento tal cual, si no que es un punto del programa al que podremos saltar de manera incondicional a través de la instrucción adecuada.
- El segundo tipo es denominado subrutina. Este empieza con una etiqueta y acaba con la instrucción RETURN o RETLW, que veremos más adelante.

Deberemos tener en cuenta:

- La etiqueta es el primer campo en una línea en lenguaje ensamblador y puede no existir.
- Si una etiqueta está presente, el ensamblador la define como el equivalente a la dirección del primer byte correspondiente a esa instrucción.
- Esta etiqueta puede volver a usarse en otro lugar pero como operando de una instrucción. El ensamblador reemplazará ésta etiqueta por el valor de cuando fue creada. Se usan frecuentemente en las instrucciones de salto.
- No puede existir más de una etiqueta en la primera columna o primer campo de instrucción.
- No pueden usarse como nombres de etiquetas a palabras ya reservadas por el ensamblador (ORG, EQU, etc.) o nombres de instrucciones (movlw, call, nop, etc.)

Ejemplo:

```
DATO EQU 05h
```

```
INICIO movlw DATO
      goto INICIO
```

La instrucción **goto INICIO** causa que la dirección de la instrucción con la etiqueta **INICIO (movlw)** se cargue en el **PC (Contador de Programa)**. Por lo tanto ésta instrucción será luego ejecutada.

No se permite el uso de números o caracteres no alfabéticos como primera letra de la etiqueta. Como regla práctica: usar siempre letras, y en mayúscula, al menos la primera.

Ejemplos:

```
TABLA2X2 Permitido
+PESO NO permitido!
=>SALIDA NO permitido!
-SALTO NO permitido!
5ALFA NO permitido!
Dato1 Permitido
Dato2 Permitido
Loop_A Permitido
```

Operandos y direcciones

Los ensambladores permiten elegir con libertad el tipo de elemento a colocar en el campo de operando o dirección.

Sistemas de numeración

Los ensambladores aceptan números Hexadecimales, octales, binarios o decimal. Esta es la forma de representarlos:

Hexadecimal:

0A00h

\$0A00

Binario:

%01001011

B'00100101'

01011010b

Octal:

@123

123Q

Decimal:

D'250'

.250

Ejemplo:

movlw .100

Significa: "mover el número literal 100 en decimal al registro de trabajo W"

Ya hemos indicado que MPLAB es el entorno de desarrollo de Microchip e incluye el ensamblador MPASM, para obtener información sobre la convención utilizada por este ver MPASM, el ensamblador de Microchip

Nombres

Los nombres pueden aparecer en el campo de operando; éstos son tratados como el dato que representan (Ver directiva EQU).

Códigos de caracteres

Algunos ensambladores permiten el uso de caracteres en ASCII. Por ejemplo:

data "hola 1,2,3" ;cadena de caracteres

data 'N' ;carácter sencillo

CHAR equ 't'

movlw 'R'

Expresiones lógicas y aritméticas

Los ensambladores permiten combinaciones de datos con operandos especiales, aritméticos o lógicos. Éstos operandos se llaman expresiones.

Por ejemplo:

REG1 EQU 05h

VALOR EQU 20h

movlw VALOR+2

addwf REG1,1

addwf REG1+1,1

En estos casos el compilador utilizará el resultado de sumar (VALOR+2) o (REG+1) como operando.

Directivas del ensamblador

Las instrucciones que podemos utilizar con un dispositivo son las que proporciona el fabricante para su producto y que forman parte del llamado "*repertorio de instrucciones*". Pero al utilizar un programa ensamblador podemos introducir además instrucciones o comando que proporciona el propio ensamblador. Estos comandos generalmente se utilizan para simplificar la tarea de programar, y reciben el nombre de directivas.

Por lo tanto las directivas no se traducen directamente a instrucciones del lenguaje máquina sino que asignan al programa ciertas áreas de memoria, definen símbolos, designan áreas de RAM para almacenamiento de datos temporales, colocan tablas o datos constantes en memoria y permiten referencias a otros programas.

Las directivas se utilizan como comandos escritos en el código fuente para realizar un control directo o ahorrar tiempo a la hora de ensamblar. El resultado de incorporar directivas se puede ver en el fichero *.LST, después de ensamblar el programa.

Para usar estas directivas o pseudo-operandos, el programador las coloca en el campo del código de operación, y, si lo requiere la directiva, una dirección o dato en el campo de dirección.

Hay que aclarar que las instrucciones de los PIC's son únicas y que no hay nada más, por ejemplo en el PIC16F84A son sólo 35 (ver instrucciones del PIC16F84A). Esto debe tenerse claro porque cuando se comienza con el ensamblador pueden confundirse un poco las propias instrucciones de los PIC's con las directivas propias del ensamblador.

A continuación se exponen las más relevantes.

Directiva EQU

El nombre viene de la palabra "equal", (igual)". La directiva EQU permite al programador "igualar" nombres personalizados a datos o direcciones. Los nombres utilizados se refieren generalmente a direcciones de dispositivos, datos numéricos, direcciones de comienzo, direcciones fijas, posiciones de bits, etc. Un nombre es más descriptivo que una simple dirección y la tarea de programar se hará mucho más sencilla. También podemos asignar un nombre a una instrucción que repitamos varias veces a lo largo de un algoritmo, de manera que sea mucho más sencilla la programación. A estos nombres que asignamos mediante esta directiva se les denomina constantes, ya que el registro al que apuntan no variará durante el programa.

Ejemplos:

```
temp equ 12
DATO EQU 22
PORT_A EQU 5
START EQU 0
CARRY EQU 3
TIEMPO EQU 5
Bank_1 EQU BSF STATUS,RP0
```

Estas líneas también pueden estar incluidas en un archivo aparte al ASM (véase directiva INCLUDE).

No siempre es necesario que con esta directiva se igualen posiciones de memoria a las etiquetas, ya que podemos poner nombres a datos. Podemos definir una equivalencia con el nombre de otra equivalencia ya definida y realizar operaciones matemáticas. Por ejemplo, podemos calcular la frecuencia del ciclo máquina a partir de la frecuencia de reloj con la finalidad de emplearla para hacer otros cálculos de la manera que se describe a continuación:

```
PORT_B EQU PORT_A+1
PORT_C EQU PORT_A+2
FIN EQU START+100
FIN2 EQU START+200
clockrate EQU .4000000 ;frecuencia del cristal
fclk EQU clockrate/4 ;frecuencia del reloj interno
```

El valor del operando debe estar ya definido anteriormente, sino el compilador entregará un error.

Además de esto, podemos igualar a las etiquetas cualquier otro tipo de valores que usemos, como, por ejemplo, el cero y el 1 en el bit de destino:

```
W EQU 0
F EQU 1
```

Con esto último, cuando usemos una instrucción donde debamos especificar donde se almacenará el resultado, en **w** o en un registro, en lugar de escribir :

- 1: para que el resultado se almacene en **f**.
- 0: para que el resultado se almacene en **w**.

Pondremos:

- F: para que el resultado se almacene en **f**.
- W: para que el resultado se almacene en **w**.

Generalmente esto último no será necesario realizarlo, siempre que incluyamos el fichero "INC" correspondiente al PIC con el que estemos trabajando (véase directiva INCLUDE).

Directiva ORG

Esta directiva dice al ensamblador a partir de que posición de memoria de programa se situarán las siguientes instrucciones. Rutinas de comienzo, subrutinas de interrupción y otros programas deben comenzar en locaciones de memoria fijados por la estructura del microcontrolador. Recordemos que el 16F84 sólo tiene 1024 posiciones de memoria flash para código.

La directiva ORG hace al compilador colocar el código que le sigue en una nueva dirección de memoria (la salida del compilador no solo coloca los códigos de operación sino también las direcciones de cada instrucción del programa). Usualmente se la utiliza para: reset, programas de servicios de interrupción, programa principal, subrutinas.

Ejemplos:

1) Inicia el programa en la posición cero:

```
ORG    0x00
```

2) Inicia el programa en la posición 0000h y luego pasa a la 0005h para no utilizar la posición del vector de interrupción (0004 h)

```
    ORG    0x00    ; El programa comienza en la dirección 0 y
```

```
    GOTO   inicio  ; salta a la dirección 5 para sobrepasar
```

```
    ORG    0x05    ; el vector de interrupción, situado en la posición 4
```

```
Inicio  xxx...
```

3) Inicia el programa en la posición 0000h y luego pasa a la 0005h para no utilizar la posición del vector de interrupción (0004 h). Si se produce una interrupción se pasa a la posición **interr**. Las subrutinas comienzan a partir de la dirección 0300h.

```
    ORG  00h    ;vector de reset
```

```
    goto inicializa
```

```
    ORG  04h    ;vector de interrupción
```

```
    goto interr
```

```
    ORG  05h
```

```
inicializa  movlw 08h    ;aquí comienza el programa
```

```
    .
```

```
    .
```

```
    ORG  300h    ;subrutinas
```

```
Subrutina1
```

```
    .
```

```
    .
```

```
    return
```

```
Subrutina2
```

```
    .
```

```
    .
```

```
    return
```

Directiva #INCLUDE

Esta directiva indica que archivos deberán tomarse en cuenta a la hora de compilar el código. Normalmente se usa para incluir el archivo de PIC que el ensamblador tiene entre sus archivos, con el cual el compilador será capaz de reconocer todos los registros especiales y sus bits. Su uso nos recordará al #include del lenguaje C. Esta línea debe colocarse al principio, y tiene la siguiente sintaxis:

#INCLUDE ; Lista de etiquetas de microchip

En ciertas ocasiones gran cantidad errores son debidos a que el nombre del archivo puesto entre comillas no se escribe correctamente.

Si utilizamos MPLAB, un entorno de desarrollo que proporciona gratuitamente Microchip, dispondremos de los archivos con extensión **.INC** para cada uno de los PIC desarrollados hasta la aparición de la versión de **MPLAB** que utilicemos. En estos archivos se definen todos los registros así como otros elementos de acuerdo al microcontrolador que estemos utilizando.

También podemos crear nuestros propios archivos "INC" con funciones, definiciones y subrutinas que utilicemos a menudo en nuestro código para evitar tener que copiarlas cada vez.

El archivo P16F84A.INC que viene con MPLAB contiene definiciones de registros, bits y bits de configuración. Los archivos INC pueden verse con cualquier editor de texto pero no se recomienda modificarlos, para no perder compatibilidad con programas desarrollados por otros.

Utilizar el **INC** del PIC que estamos utilizando en nuestro programa no es obligatorio, y podemos omitirlo, pero a cambio tendremos que definir los nombres de los registros que usemos o bien llamarlos por su posición de memoria.

Esto puede a la larga ser problemático de manera que se recomienda utilizar los archivos **INC** correspondientes al PIC que utilicemos porque además de facilitar la creación del programa al no tener que recordar las direcciones reales de los registros también se facilita el paso de un programa diseñado para un microcontrolador hacia otro distinto.

Si utilizamos las posiciones de memoria con la dirección real, podemos hacer incompatibles las operaciones entre registros. Por ejemplo, CLRF 0x05, borra el registro ubicado en esa dirección, que no es ni más ni menos que el PORTA (Puerto A) en el PIC16F84A. Pues bien, si queremos actualizarnos a otro microcontrolador pero resulta que en este el registro 0x05 tiene otra función nos será mucho más difícil actualizar el programa. Ahora bien, si hubiésemos utilizado CLRF PORTA, y el .INC correspondiente al nuevo microcontrolador ya se ocupará el ensamblador de realizar las correspondencias.

Y por supuesto siempre será más fácil recordar PORTA que no 0x05.

También permite incluir otros programas. Por ejemplo:

```
#INCLUDE "DISPLAY.ASM"
```

Esto le dice al compilador que incluya el código que se encuentra en el archivo DISPLAY.ASM como si fuese parte del propio programa. Esto es muy útil para reutilizar códigos realizados con anterioridad.

Directiva LIST

Este comando sirve para que el compilador tenga en cuenta sobre qué procesador se está trabajando. Este comando debe estar en todo proyecto, situado debajo del "include", con la siguiente sintaxis.

```
LIST P=PIC16F84A
```

Directiva END

Al igual que las dos anteriores, esta debe ir incluida una sola vez en todo el programa. En concreto, esta debe situarse al final, para indicar al ensamblador que el programa ha finalizado. Esta siempre debe estar presente, aunque el flujo de nuestro programa acabe en un bucle.

Directiva #DEFINE

#DEFINE es una directiva muy útil. Define se usa para crear pequeñas macros. Con estas macros podremos poner nombres a pequeños fragmentos de código que nos facilitarán la realización y comprensión del algoritmo.

Por ejemplo, podremos poner nombres a bits.

```
#define CERO STATUS,2
```

Así, en vez de tener que llamar al bit por un número y un registro, podremos usar directamente la palabra CERO.

```
#define CINCO 5
```

Cada vez que se utilice la palabra **CINCO** será reemplazada en el momento de la compilación por el número 5.

Otro ejemplo muy práctico es el de poner nombre a un fragmento de código usado frecuentemente. Este fragmento de código, puede ser por ejemplo, el que conmuta entre los dos bancos.

```
BSF    OPTION,RP0
```

```
BCF    OPTION,RP0
```

Como cambiamos varias veces de banco a lo largo de un algoritmo, puede resultar más práctico ponerle un nombre.

```
#define BANCO1 BSF OPTION,RP0
```

```
#define BANCO0 BCF OPTION,RP0
```

De este modo bastará con poner BANCO1 o BANCO0 para conmutar entre los dos bancos de memoria de manera que cada vez que se utilice la palabra **BANCO1**, en realidad se estará utilizando la instrucción **BSF STATUS,RP0**

En el siguiente ejemplo:

```
#define salida PORTA,3
```

No tendremos necesidad de recordar cuál era la patilla de salida, sino que solo lo mencionaremos como **salida**. Cada vez que aparezca la palabra **salida** en el código, ésta será interpretada como PORTA,1 que es una instrucción válida. Podemos ponerlo a cero con la instrucción.

```
BCF salida
```

En vez de tener que poner.

```
BCF PORTA,3
```

Una cosa a tener en cuenta es que con la directiva INCLUDE, podemos prescindir del carácter almohadilla (#), pero en el caso de la directiva DEFINE, no.

Esta directiva es muy útil porque hace el código más fácil de leer y entender.

Directiva TITLE

Esta directiva no sirve de mucho, pero será útil para aquellos que quieran que el compilador tenga en cuenta el título que le ha puesto a su código. Tiene la siguiente sintaxis:

```
TITLE "Nombre del código"
```

Este nombre aparecerá en los archivos .lst (listados) que cree el compilador.

Directivas IF...ELSE...ENDIF

Algunos ensambladores permiten incluir o excluir partes del programa dependiendo de condiciones que existan en el tiempo de compilación.

La forma típica es:

IF CONDICION

.

ELSE

.

ENDIF

Ejemplo:

SINK EQU 1 ; (cambiar por 0 en caso necesario)

IF SINK=1

BCF PORTA,0

ELSE

BSF PORTA,0

ENDIF

En este caso el valor de **SINK** hará que el compilador utilice distintas instrucciones de código.

Si la condición es verdadera en el tiempo de compilación, las instrucciones que están entre IF y ELSE se incluirán en el programa. Si la condición es falsa se incluirán en el programa las instrucciones entre ELSE y ENDIF.

Los usos típicos son:

- Para incluir o excluir variables extras
- Para incluir código de diagnóstico en condiciones de testeo (DEBUG).
- Para permitir datos de distintos tamaños.

Desgraciadamente, el ensamblado condicional, tiende a complicar la lectura del programa, por lo tanto, sólo debemos utilizarlo si es necesario.

Directiva MACRO

Esta directiva resulta muy potente y a diferencia de la directiva #define se pueden crear macros más extensas, lo que nos evitará tener que ejecutar reiteradamente fragmentos de código idénticos. Cuando una macro es invocada, esta es copiada por el ensamblador en el lugar de la invocación dentro del código fuente. La macro se declara con la directiva MACRO, y termina con la directiva ENDM.

Creación de una macro denominada **activar**:

activar macro

CLRF PORTA

BSF PORTB,2

endm

Hemos creado una macro llamada **activar** de manera que en nuestro código cada vez que pongamos la palabra **activar**, el ensamblador la reemplazará por **CLRF PORTA...** etc. hasta el final de la macro que termina con la directiva **ENDM** (fin macro).

Las macros permiten asignar un nombre a una secuencia de instrucciones de manera que son útiles cuando ocurren secuencias de instrucciones repetitivas. Luego se utiliza el nombre de la macro en el programa como si se usara la secuencia de instrucciones anterior.

Las macros no son lo mismo que las subrutinas. El código de las subrutinas aparece una sola vez en un programa y la ejecución del programa salta a la subrutina. En cambio, el ensamblador reemplaza cada aparición del nombre de la macro con la secuencia especificada de instrucciones. Por consiguiente la ejecución del programa no salta a la macro como una subrutina.

Ejemplo:

Archivo "MULX10.ASM"

```
MULX10 MACRO          ;comienzo de la macro
    MOVF  tiempo,W    ;guarda el tiempo en W
    RLF   tiempo      ;multiplica por 2
    RLF   tiempo      ;multiplica por 2
    RLF   tiempo      ;multiplica por 2
    ADDWF tiempo      ;le suma una vez más
    ADDWF tiempo      ;le suma una vez más
    ENDM              ;fin de la macro
```

Archivo "EJEMPLO1.ASM":

```
#INCLUDE "MULX8.ASM"
tiempo EQU      0Ch
resultado EQU    0Dh

    MOVLW 20
    MOVWF tiempo
```

```

MULX10
MOVWF resultado
END

```

Si ensamblamos "EJEMPLO1.ASM" notaremos que el listado final (EJEMPLO.LST) queda de la siguiente forma:

```

tiempo EQU 0Ch
resultado EQU 0Dh

    MOVLW 20
    MOVWF tiempo
    MOVF tiempo,W ;guarda el tiempo en W
    RLF tiempo ;multiplica por 2
    RLF tiempo ;multiplica por 2
    RLF tiempo ;multiplica por 2
    ADDWF tiempo ;le suma una vez más
    ADDWF tiempo ;le suma una vez más
    MOVWF resultado
END

```

Problemas con las MACROS

Con las macros se puede trabajar rápidamente, pero pueden resultar poco eficientes. Veamos un error muy común al utilizar macros, en este caso se utiliza una macro denominada MOVFF:

```

MULX10 MACRO ;comienzo de la macro
    MOVF AUX1,W ;Mueve contenido de un registro a otro
    MOVWF AUX2 ;a través del acumulador
ENDM ;fin de la macro

```

Porción de código:

```

MOVLW .1 ;TEMP=1
MOVWF TEMP
DECF TEMP,F ;Z se va a 1
BTFSS STATUS,Z ;salta si o si
MOVFF AUX1,AUX2 ;Macro
MOVWF PORTA
; ...

```

En la línea de la macro está el error porque los saltos (BTFSS) no pueden saltar macros. Las macros están compuestas por más de una instrucción, y el salto se producirá dentro de la misma.

El código anterior con la macro incrustada sería:

```

MOVLW .1 ;TEMP=1
MOVWF TEMP
DECF TEMP,F ;Z se va a 1
BTFSS STATUS,Z ;salta si o si
MOVF AUX1,W ;líneas de anterior macro
MOVWF AUX2 ;
MOVWF PORTA
; ...

```

Otro tema importante, que se ilustra en este ejemplo, es que las macros pueden modificar registros (en este caso W) de forma que el programador podría no tener en cuenta.

En el ejemplo anterior, PORTA se debería cargar con 1, que aparentemente era el valor de W, pero la macro lo ha modificado, lo que resulta en otro error.

Ejemplos de macros

```

; *****
; macros.asm ;
; "MACROS para 16F84" ;
; *****
;

```

```

callz macro subrutina
    btfsc STATUS,Z
    call subrutina
endm

```

```

callnz macro subrutina
    btfss STATUS,Z
    call subrutina
endm

```

```

    endm

movff macro f2,f1 ;(atención, se destruye W)
    movf f1,w
    movwf f2
endm

movlf macro file,literal ;(atención, se destruye W)
    movlw literal
    movwf file
endm

;Atención, para usar estas macros ya debe estar activo el banco 1
CONF_PORTA macro dato
    movlw dato
    movwf TRISA
endm

CONF_PORTB macro dato
    movlw dato
    movwf TRISB
endm

;configurar Option Register:
CONF_OPTION macro dato
    movlw dato
    movwf OPTION_REG
endm

;configurar el registro de interrupciones:
CONF_INTCON macro dato
    movlw dato
    movwf INTCON
endm

SET_BANK_0 macro
    BCF STATUS,RP0
endm

SET_BANK_1 macro
    BSF STATUS,RP0
endm

;enable y disable all the mascarable interrupts (16F84):
EI macro
    bsf INTCON,GIE
endm

DI macro
    bcf INTCON,GIE
endm

#define iEnable EI
#define iDisable DI

;arrancar el timer:
RESET_TIMER macro
    bcf INTCON,T0IF
endm

; inicializar timer antes de hacer RESET_TIMER para que arranque.
INIT_TIMER macro dato
    movlw dato
    movwf TMR0
endm

jmp macro salto
    goto salto

```

```

        endm

ret      macro
        return
        endm

;Complemento a 1 de W:
comw     macro
        xorlw 0xff
        endm

;Instrucciones de salto tipo Z80

jz       macro _salto      ;salta si zero
        btfsc STATUS,Z
        goto  _salto
        endm

jnz      macro _salto      ;salta si no zero
        btfss STATUS,Z
        goto  _salto
        endm

jc       macro _salto      ;salta si carry
        btfsc STATUS,C
        goto  _salto
        endm

jnc      macro _salto      ;salta si no carry
        btfss STATUS,C
        goto  _salto
        endm

. *****
.
.         FIN
. *****
.

```

Instrucciones del PIC16F84A

Introducción

El PIC16F84A pertenece a la gama media y es de tipo RISC; esto quiere decir que tiene un juego de instrucciones reducido, en concreto de 35 instrucciones o nemónicos que son la base de funcionamiento del PIC. Al igual que los bits de los registros, sería complicado memorizarlas todas, así que utilizaremos este documento como guía de consulta.

Las instrucciones fundamentalmente se dividen en tres tipos. Esta división viene dada por el tipo de datos con los que trabajan:

- Instrucciones orientadas a los registros o bytes (byte-oriented operations).
- Instrucciones orientadas a los bits (bit-oriented operations).
- Operaciones con literales y de control (literal and control operations).

Repertorio 35 instrucciones

Las 35 instrucciones ó mnemónicos de la gama media de Microchip las encontraremos resumidas en la siguiente tabla. **w** es el acumulador, **f** representa un registro cualquiera y **C**, **DC**, **Z** los flags del registro **STATUS**.

Instrucciones orientadas a registros						
MNEMÓNICO OPERANDOS		DESCRIPCIÓN	CÓDIGO OP	BANDERAS	NCIC	NOTAS
ADDWF	f,d	w + f → d	00 0111 dfff ffff	C, DC, Z	1	1,2
ANDWF	f,d	w AND f → d	00 0101 dfff ffff	Z	1	1,2
CLRF	f	00 h → f	00 0001 1fff ffff	Z	1	2
CLRW	-	00 h → w	00 0001 0xxx xxxx	Z	1	-
COMF	f,d	Complemento de f → d	00 1001 dfff ffff	Z	1	1,2
DECF	f,d	f - 1 → d	00 0011 dfff ffff	Z	1	1,2
DECFSZ	f,d	f - 1 → d (si es 0 salta)	00 1011 dfff ffff	Ninguna	1(2)	1,2,3
INCF	f,d	f + 1 → d	00 1010 dfff ffff	Z	1	1,2
INCFSZ	f,d	f + 1 → d (si es 0 salta)	00 1111 dfff ffff	Ninguna	1(2)	1,2,3
IORWF	f,d	w OR f → d	00 0100 dfff ffff	Z	1	1,2
MOVF	f,d	f → d	00 1000 dfff ffff	Z	1	1,2
MOVWF	f	w → f	00 0000 1fff ffff	Ninguna	1	-
NOP	-	No operación	00 0000 0xx0 0000	Ninguna	1	-
RLF	f,d	Rota f izq por carry → d	00 1101 dfff ffff	C	1	1,2
RRF	f,d	Rota f dcha por carry → d	00 1100 dfff ffff	C	1	1,2
SUBWF	f,d	f - w → d	00 0010 dfff ffff	C,DC,Z	1	1,2
SWAPF	f,d	Intercambia nibbles de f → d	00 1110 dfff ffff	Ninguna	1	1,2
XORWF	f,d	w XOR f → d	00 0110 dfff ffff	Z	1	1,2

Instrucciones orientadas a bit						
MNEMÓNICO OPERANDOS		DESCRIPCIÓN	CÓDIGO OP	BANDERAS	NCIC	NOTAS
BCF	f,b	Pone a 0 bit b de registro f	01 00bb bfff ffff	Ninguna	1	1,2
BSF	f,b	Pone a 1 bit b de registro f	01 01bb bfff ffff	Ninguna	1	1,2

BTFSC	f,b	Salto si bit b de reg. f es 0	01 10bb bfff ffff	Ninguna	1(2)	3
BTFSS	f,b	Salto si bit b de reg. f es 1	01 11bb bfff ffff	Ninguna	1(2)	3

Instrucciones con literales y de control						
MNEMONICO OPERANDOS		DESCRIPCIÓN	CÓDIGO OP	BANDERAS	NCIC	NOTAS
ADDLW	k	$w + k \rightarrow w$	11 111x kkkk kkkk	C,DC,Z	1	-
ANDLW	k	$w \text{ AND } k \rightarrow w$	11 1001 kkkk kkkk	Z	1	-
CALL	k	Llamada a subrutina k	10 0kkk kkkk kkkk	Ninguna	2	-
CLRWDI	-	Borra temporizador del WDI	00 0000 0110 0100	TO,PD	1	-
GOTO	k	Ir a dirección k	10 1kkk kkkk kkkk	Ninguna	2	-
IORLW	k	$w \text{ OR } k \rightarrow w$	11 1000 kkkk kkkk	Z	1	-
MOVLW	k	$k \rightarrow w$	11 00xx kkkk kkkk	Ninguna	1	-
RETFIE	-	Retorno de una interrupción	00 0000 0000 1001	Ninguna	2	-
RETLW	k	Retorno con k en w	11 01xx kkkk kkkk	Ninguna	2	-
RETURN	-	Retorno de una subrutina	00 0000 0000 1000	Ninguna	2	-
SLEEP	-	Modo Standby	00 0000 0110 0011	TO, PD	1	-
SUBLW	k	$k - w \rightarrow w$	11 110x kkkk kkkk	C,DC,Z	1	-
XORLW	k	$w \text{ XOR } k \rightarrow w$	11 1010 kkkk kkkk	Z	1	-

Notas:

1. Al modificar un registro de E/S con una operación sobre él mismo (por ejemplo MOVF PORTB,1), el valor utilizado es el que se halle presente en las patillas del PORTB. Por ejemplo, si el biestable tiene un "1" para una patilla configurada como entrada y se pone a nivel bajo desde el exterior, el dato se volverá a escribir como "0".
2. Si se ejecuta esta instrucción sobre el TMR0 y d=1, se borrará el conteo de la preescala asignada (preescaler), si está asignado al TMR0, pero no se borrará la preescala asignada en OPTION_REG, que controla Timer0.
3. Si se modifica el Contador de Programa **PC** o una condición de prueba es verdadera, la instrucción requiere dos ciclos máquina. El segundo ciclo se ejecuta como un NOP.

En las tablas siguientes, por orden alfabético, veremos todos los datos de interés sobre las 35 instrucciones.

Algunos son de poca importancia. En cambio otros, como la operación, la sintaxis, el comportamiento del registro STATUS y los ejemplos, son imprescindibles para comprender su funcionamiento.

Aparte de estas 35 instrucciones, hay otro tipo de instrucciones usadas para simplificar la tarea de programar, y que generalmente están formadas por dos instrucciones básicas. Estas no las trataremos a fondo, pero las veremos en un resumen después de comprender el funcionamiento de las 35 instrucciones básicas.

ADDLW					ADDLW
ADD Literal to w					
Operación	$w + k \rightarrow w$				
Sintaxis	[Etiqueta] ADDLW <i>k</i>				
Operadores	$0 < k < 255$				
Ciclos	1				
OPCODE	11	111x	kkkk	kkkk	
Descripción	Suma el contenido del registro w al literal k , y almacena el resultado en w . Si se produce acerreo el flag C se pone a "1".				

Registro de **STATUS**

PA2	PA1	PA0	TO#	PD#	Z	DC	C
-	-	-	-	-	X	X	X

C Se pone a 1 si se produce un Acarreo desde el bit de mayor peso.

DC Se pone a 1 si se genera un Acarreo del bit 3 al bit 4.

Z Se pone a 1 si el resultado de la operación es cero.

EJEMPLO:

ADDLW 0x15

Si antes de la instrucción:

w = 10h = 0001 0000 b

Al ejecutarse la instrucción

w = 10 h + 15 h = 25 h

w = 0001 0000 b + 0001 0101 b = 0010 0101 b

0001 0000 b

0001 0101 b

0010 0101 b

ADDWF					ADDWF
ADD w to F					
Operación	w + f → d				
Sintaxis	[Etiqueta] ADDWF f,d				
Operadores	0 < f < 127 d [0,1]				
Ciclos	1				
OPCODE	00	0111	dfff	ffff	
Descripción	Suma el contenido del registro w al contenido del registro f , y almacena el resultado en w si d = 0, y en el registro f si d = 1.				

Registro de **STATUS**

PA2	PA1	PA0	TO#	PD#	Z	DC	C
-	-	-	-	-	X	X	X

C Se pone a 1 si se produce un Acarreo desde el bit de mayor peso

DC Se pone a 1 si se genera un Acarreo del bit 3 al bit 4.

Z Se pone a 1 si el resultado de la operación es cero

EJEMPLO:

ADDWF FSR,0

Si antes de la instrucción. $w = 17\text{ h}$ y $\text{FSR} = \text{C2 h}$ como $d=0$

Al ejecutarse:

$w = 17\text{ h} + \text{C2 h} = \text{D9 h}$

$\text{FSR} = \text{C2 h}$

0001 0111 b

1100 0010 b

1101 1001 b

Volver a tabla

ANDLW				ANDLW
AND Literal and w				
Operación	w AND k → w			
Sintaxis	[Etiqueta] ANDLW k			
Operadores	0 < f < 255			
Ciclos	1			
OPCODE	11	1001	kkkk	kkkk
Descripción	Efectúa la operación AND lógico entre el contenido del registro w y el literal k , y almacena el resultado en w . Esta instrucción realiza la operación AND bit a bit.			

Registro de **STATUS**

PA2	PA1	PA0	TO#	PD#	Z	DC	C
-	-	-	-	-	X	-	-

Z Se pone a 1 si el resultado de la operación es cero

EJEMPLO:

ANDLW 0x5F

Si antes de la instrucción. $w = A3 \text{ h}$

Al ejecutarse:

$w = 0101 \ 1111 \text{ b AND } 1010 \ 0011 \text{ b} = 0000 \ 0011 \text{ b} = 03 \text{ h}$

0101 1111 b

1010 0011 b

0000 0011 b

ANDWF				ANDWF
AND w with F				
Operación	w AND f → d			
Sintaxis	[Etiqueta] ANDWF f,d			
Operadores	0 < f < 127 d [0,1]			
Ciclos	1			
OPCODE	00	0101	dfff	ffff
Descripción	Efectúa la operación AND lógico entre el contenido del registro w y el contenido del registro f , y almacena el resultado en w si d = 0 , y en f si d = 1 . Esta instrucción realiza la operación AND bit a bit.			

Registro de **STATUS**

PA2	PA1	PA0	TO#	PD#	Z	DC	C
-	-	-	-	-	X	-	-

Z Se pone a 1 si el resultado de la operación es cero

EJEMPLO:

ANDWF FSR,1

Si antes de la instrucción. w = 17 h = 0001 0111 b y FSR = C2 h = 1100 0010 h

Al ejecutarse:

w = 17 h = 0001 0111 b

FSR = 0001 0111 b AND 1100 0010 b = 0000 0010 b = 02 h

0001 0111 b

1100 0010 b

0000 0010 b

BCF				BCF
Bit Clear F				
Operación	0 → (f)			
Sintaxis	[Etiqueta] BCF f,b			
Operadores	0 < f < 127 0 < b < 7			
Ciclos	1			
OPCODE	01	00bb	bfff	ffff
Descripción	Pone a cero el bit número b del registro f .			

Registro de **STATUS**

PA2	PA1	PA0	TO#	PD#	Z	DC	C
-	-	-	-	-	-	-	-

EJEMPLO:

BCF FLAG_REG, 7

Si antes de la instrucción el registro:

FLAG_REG = C7 h = 1100 0111 b

Al ejecutarse la instrucción, el registro queda con el valor:

FLAG_REG = 47b = 0100 0111 b

BSF				BSF
Bit Set F				
Operación	1 → (f)			
Sintaxis	[Etiqueta] BSF f,b			
Operadores	0 < f < 127 0 < b < 7			
Ciclos	1			
OPCODE	01	11bb	bfff	ffff
Descripción	Pone a 1 el bit b del registro f			

Registro de **STATUS**

PA2	PA1	PA0	TO#	PD#	Z	DC	C
-	-	-	-	-	-	-	-

EJEMPLO:

BSF FLAG_REG, 7

Si antes de la instrucción el registro tiene el valor:

FLAG_REG = 0A h = 0000 1010 b

Al ejecutarse la instrucción, el registro queda con el valor:

FLAG_REG = 8A h = 1000 1010 b

BTFSC					BTFSC
Bit Test, Skip if Clear					
Operación	Salta si (f < b >) = 0				
Sintaxis	[Etiqueta] BTFSC f,b				
Operadores	0 < f < 127 0 < b < 7				
Ciclos	1 (2)				
OPCODE	01	10bb	bfff	ffff	
Descripción	Si el bit número b del registro f es cero, la instrucción que sigue a ésta se ignora y se trata como un NOP (skip). En este caso, y sólo en este caso, la instrucción BTFSC precisa dos ciclos para ejecutarse.				

Registro de **STATUS**

PA2	PA1	PA0	TO#	PD#	Z	DC	C
-	-	-	-	-	-	-	-

EJEMPLO:

INICIO BTFSC FLAG,1

ES_1 GOTO PROCESO

ES_0

Si antes de la instrucción. PC = dirección INICIO

Al ejecutarse:

if FLAG<1> = 0,

PC = dirección ES_0 y seguirá la ejecución del programa.

if FLAG<1> = 1,

PC = dirección ES_1 y el programa continuará en PROCESO

BTFSS					BTFSS	
Bit Test, Skip if Set						
Operación	Salta si (f < b >) = 1					
Sintaxis	[Etiqueta] BTFSS f,b					
Operadores	0 < f < 127 0 < b <7					
Ciclos	1 (2)					
OPCODE	01	11bb	bfff	ffff		
Descripción	Si el bit número b del registro f está a 1, la instrucción que sigue a ésta se ignora y se trata como un NOP (skip). En este caso, y sólo en este caso, la instrucción BTFSS precisa dos ciclos para ejecutarse.					

Registro de STATUS

PA2	PA1	PA0	TO#	PD#	Z	DC	C
-	-	-	-	-	-	-	-

EJEMPLO:

INICIO BTFSS FLAG,1
ES_0 GOTO PROCESO
ES_1

Si antes de la instrucción. PC = dirección INICIO

Al ejecutarse:

if FLAG<1> = 0,

PC = dirección ES_0 y el programa continuará en PROCESO.

if FLAG<1> = 1,

PC = dirección ES_1 y seguirá la ejecución del programa.

CALL					CALL
Subrutine Call					
Operación	PC + 1 → TOS k → PC <10:0> PCLATCH (<4:3>) → PC (<12,11>)				
Sintaxis	[Etiqueta] CALL k				
Operandos	0 = k = 2047				
Ciclos	2				
OPCODE	10	0kkk	kkkk	kkkk	
Descripción	Salvaguarda la dirección de vuelta en la Pila y después llama a la subrutina situada en la dirección cargada en el PC. El modo de cálculo de la dirección efectiva difiere según la familia PIC utilizada. También hay que posicionar PA2, PA1 y PA0 (PIC 16C5X) o el registro PCLATCH (En los demás PIC) antes de ejecutarse la instrucción.				

Registro de **STATUS**

PA2	PA1	PA0	TO#	PD#	Z	DC	C
-	-	-	-	-	-	-	-

EJEMPLO:

INICIO CALL SUB_1

Si antes de la instrucción:

PC = dirección INICIO

Al ejecutarse:

PC = dirección SUB_1

TOS = dirección INICIO +1

CLRF				CLRF
Clear f				
Operación	00 h → f 1 → Z			
Sintaxis	[Etiqueta] CLRF f			
Operadores	0 < f < 127			
Ciclos	1			
OPCODE	00	0001	1fff	ffff
Descripción	Se borra el contenido del registro f y el flag Z se activa			

Registro de **STATUS**

PA2	PA1	PA0	TO#	PD#	Z	DC	C
-	-	-	-	-	1	-	-

Z Se pone a 1 si el resultado de la operación es cero

EJEMPLO:

CLRf REG

Si antes de la instrucción:

REG = 5A h

Al ejecutarse:

REG = 00 h

flag Z = 1

CLRw				CLRw
Clear w				
Operación	00 h → w 1 → Z			
Sintaxis	[Etiqueta] CLRw			
Operadores	No tiene			
Ciclos	1			
OPCODE	00	0001	0000	0011
Descripción	El registro de trabajo w se carga con 00h. El flag Z se pone a 1			

Registro de **STATUS**

PA2	PA1	PA0	TO#	PD#	Z	DC	C
-	-	-	-	-	1	-	-

Z Se pone a 1 si el resultado de la operación es cero

EJEMPLO

CLRw

Si antes de la instrucción. w= 5Ah

Al ejecutarse:

w = 00

flag Z = 1

CLRWDT					CLRWDT
Clear watchdog Timer					
Operación	00 h → WDT 1 → T0# 1 → PD#				
Sintaxis	[Etiqueta] CLRWDT				
Operadores	No tiene				
Ciclos	1				
OPCODE	00	0000	0110	0100	
Descripción	Se borra tanto el registro WDT (watchdog) como su preescaler. Los bits T0# y PD# del registro de estado se ponen a "1".				

Registro de **STATUS**

PA2	PA1	PA0	T0#	PD#	Z	DC	C
-	-	-	1	1	-	-	-

T0# Se pone a 1 cuando se ejecuta la instrucción CLRWDT o SLEEP. Se pone a 0 si el temporizador watchdog se desborda

PD# Se pone a 1 cuando se ejecuta la instrucción CLRWDT o SLEEP

EJEMPLO

CLRWDT

Si antes de ejecutarse la instrucción

WDT = ?

Al ejecutarse: WDT = 00 h

Preescaler WDT = 0

bit de estado T0 = 1

bit de estado PD = 1

COMF				COMF
Complement f				
Operación	Complemento de f → d			
Sintaxis	[Etiqueta] COMF f,d			
Operadores	0 < f < 127 d [0,1]			
Ciclos	1			
OPCODE	00	1001	dfff	ffff
Descripción	Hace el complemento del contenido del registro f bit a bit. El resultado se almacena en el registro f si d=1 y en el registro w si d=0 , en este caso f no varía.			

Registro de **STATUS**

PA2	PA1	PA0	TO#	PD#	Z	DC	C
-	-	-	-	-	X	-	-

Z Se pone a 1 si el resultado de la operación es cero

EJEMPLO:

COMF REG1,0

Si antes de la instrucción:

REG1 = 13 h como d= 0

Al ejecutarse:

REG1 = 13 h = 0001 0011 b

w = EC h = 1110 1100 b

flag Z = 0

0001 0011 b

1110 1100 b

DECF				DECF
Decrement f				
Operación	f - 1 → d			
Sintaxis	[Etiqueta] DECF f , d			
Operadores	0 < f < 127 d [0,1]			
Ciclos	1			
OPCODE	00	0011	dfff	ffff
Descripción	Se decrementa el contenido del registro f en una unidad. El resultado se almacena en f si d=1 y en w si d=0 , en este caso f no varía.			

Registro de **STATUS**

PA2	PA1	PA0	TO#	PD#	Z	DC	C
-	-	-	-	-	X	-	-

Z Se pone a 1 si el resultado de la operación es cero

EJEMPLO:

DECF CNT,1

Si antes de la instrucción:

CNT = 01 h

Z = 0

Al ejecutarse:

CNT = 00 h

bit Z = 1

DECFSZ					DECFSZ
Decrement f , Skip if 0					
Operación	f - 1 → d, salta si resultado = 0				
Sintaxis	[Etiqueta] DECFSZ f,d				
Operadores	0 < f < 127 d [0.1]				
Ciclos	1 (2)				
OPCODE	00	1011	dfff	ffff	
Descripción	Decrementa el contenido del registro f en una unidad, el resultado se almacena en f si d=1 y en w si d=0, en este caso, f no varía. Si el resultado es cero, se ignora la siguiente instrucción y, en ese caso la instrucción tiene una duración de dos ciclos.				

Registro de **STATUS**

PA2	PA1	PA0	TO#	PD#	Z	DC	C
-	-	-	-	-	-	-	-

EJEMPLO:

INICIO DECFSZ CNT,1
 GOTO LOOP

CONTINUAR

si antes de la instrucción:

PC = dirección INICIO

Al ejecutarse: CNT = CNT -1

Si CNT = 0 entonces PC = dirección CONTINUAR

Si CNT no = 0 entonces PC = dirección INICIO + 1

GOTO					GOTO
Unconditional Branch					
Operación	$k \rightarrow PC <10:0>$ $(PCLATH <4:3>) \rightarrow (PC <12:11>)$				
Sintaxis	[Etiqueta] GOTO <i>k</i>				
Operadores	$0 < k < 2047$				
Ciclos	2				
OPCODE	10	1kkkk	kkkk	kkkk	
Descripción	Salto incondicional, normalmente se utiliza para llamar a la subrutina situada en la dirección que se carga en PC. El modo de cálculo de la instrucción carga desde el bit 0 al 10 de la constante k en el PC y los bits 3 y 4 del registro PCLATH en los 11 y 12 del PC				

Registro de **STATUS**

PA2	PA1	PA0	TO#	PD#	Z	DC	C
-	-	-	-	-	-	-	-

EJEMPLO:

GOTO SEGUIR

Al ejecutarse:

PC = dirección SEGUIR

INCF				INCF
Increment f				
Operación	f + 1 → d			
Sintaxis	[Etiqueta] INCF f,d			
Operadores	0 < f < 127 d [0,1] f + 1 → d			
Ciclos	1			
OPCODE	00	1010	dfff	ffff
Descripción	Se incrementa en una unidad el contenido del registro f , si d=1 el resultado se almacena en f , si d=0 el resultado se almacena en w , en este caso el resultado de f no varía.			

Registro de **STATUS**

PA2	PA1	PA0	TO#	PD#	Z	DC	C
-	-	-	-	-	X	-	-

Z Se pone a 1 si el resultado de la operación es cero al haber desbordamiento

EJEMPLO:

INCF CNT,1

Si antes de la instrucción:

CNT = FF h

flag Z = 0

Al ejecutarse:

FF h + 1 h = 00 h

CNT = 00

flag Z = 1

INCFSZ				INCFSZ
Increment f, SKIP if 0				
Operación	f +1 → d , salta si resultado = 0			
Sintaxis	[Etiqueta] INCFSZ f,d			
Operadores	0 < f < 127 d [0,1]			
Ciclos	1 (2)			
OPCODE	00	1111	dfff	ffff
Descripción	Incrementa el contenido del registro f en una unidad, el resultado se almacena de nuevo en f si d=1 , y en w si d=0 , en este caso, f no varía. Si el resultado es cero, se ignora la siguiente instrucción y, en ese caso la instrucción tiene una duración de dos ciclos.			

Registro de **STATUS**

PA2	PA1	PA0	TO#	PD#	Z	DC	C
-	-	-	-	-	-	-	-

EJEMPLO:

INICIO INCFSZ CNT,1
 GOTO SALTO

CONTINUAR

Si antes de la instrucción:

PC = dirección INICIO

Al ejecutarse: CNT = CNT+1

Si CNT = 0

Entonces PC = dirección CONTINUAR

Si CNT no = 0

Entonces PC = dirección INICIO + 1

IORLW				IORLW
Inclusive OR Literal with w				
Operación	w OR k → w			
Sintaxis	[Etiqueta] IORLW k			
Operadores	0 < k < 255			
Ciclos	1			
OPCODE	11	1000	kkkk	kkkk
Descripción	Se realiza la operación lógica OR entre el registro w y el literal k . El resultado se almacena en el registro w . Esta instrucción realiza la operación OR bit a bit.			

Registro de **STATUS**

PA2	PA1	PA0	TO#	PD#	Z	DC	C
-	-	-	-	-	X	-	-

Z Se pone a 1 si el resultado de la operación es cero.

EJEMPLO:

IORLW 0x35

Si antes de la instrucción:

w = 9A h

Al ejecutarse:

w = 1001 1010 b + 0011 0101 b = 1011 1111 b = BF h

1001 1010 b

0011 0101 b

1011 1111 b

IORWF					IORWF
Inclusive OR w with f					
Operación	w OR f → d				
Sintaxis	[Etiqueta] IORWF f,d				
Operadores	0 < f < 127 d [0,1]				
Ciclos	1				
OPCODE	00	0100	dfff	ffff	
Descripción	Efectúa la operación lógica OR entre el contenido del registro w y el contenido del registro f , y almacena el resultado en f si d=1 y en w si d=0 . Esta instrucción realiza la operación OR bit a bit.				

Registro de **STATUS**

PA2	PA1	PA0	TO#	PD#	Z	DC	C
-	-	-	-	-	X	-	-

Z Se pone a 1 si el resultado de la operación es cero.

EJEMPLO:

IORWF RESUL,0

Si antes de la instrucción

RESUL = 13 h = 0001 0011 b

w = 91 h = 1001 0001 b

Al ejecutarse:

RESUL= 0001 0011 b OR 1001 0001 b = 1001 0011 b = 93 h

0001 0011 b

1001 0001 b

1001 0011 b

MOVLW				MOVLW
Move literal to w				
Operación	$k \rightarrow w$			
Sintaxis	[Etiqueta] MOVLW k			
Operadores	$0 < f < 255$			
Ciclos	1			
OPCODE	11	00xx	kkkk	kkkk
Descripción	El registro w se carga con el valor de 8 bits del literal k			

Registro de **STATUS**

PA2	PA1	PA0	TO#	PD#	Z	DC	C
-	-	-	-	-	X	-	-

EJEMPLO:

MOVLW 0x5A

Al ejecutarse:

w = 5A h

MOVf				MOVf
Move f				
Operación	f → d			
Sintaxis	[Etiqueta] MOVf f,d			
Operadores	0 < f < 127 d [0,1]			
Ciclos	1			
OPCODE	00	10000	dfff	ffff
Descripción	El contenido del registro f se carga en el registro destino dependiendo del valor de d . Si d=0 el destino es el registro w , sid=1 el destino es el propio registro f . Esta instrucción permite verificar dicho registro ya que el flag Z queda afectado.			

Registro de **STATUS**

PA2	PA1	PA0	TO#	PD#	Z	DC	C
-	-	-	-	-	X	-	-

Z Se pone a 1 si el resultado de la operación es cero.

EJEMPLO:

MOVF FSR,0

Al ejecutarse:

w = al valor del FSR

MOVWF				MOVWF
Move w to f				
Operación	w → f			
Sintaxis	[Etiqueta] MOVWF f			
Operadores	0 < f < 127			
Ciclos	1			
OPCODE	00	0000	1fff	ffff
Descripción	Mueve el contenido del registro w al registro f			

Registro de **STATUS**

PA2	PA1	PA0	TO#	PD#	Z	DC	C
-	-	-	-	-	-	-	-

EJEMPLO:

MOVWF OPCION

Si antes de la instrucción:

OPCION = FF h

w = 4F h

Al ejecutarse:

OPCION = 4F h

w = 4F h

NOP				NOP
No operation				
Operación	no operación			
Sintaxis	[Etiqueta] NOP			
Operadores	No tiene			
Ciclos	1			
OPCODE	00	0000	0xx0	0000
Descripción	No realiza operación alguna, pero sirve para consumir un ciclo de instrucción, equivalente a 4 de reloj.			

Registro de **STATUS**

PA2	PA1	PA0	TO#	PD#	Z	DC	C
-	-	-	-	-	-	-	-

EJEMPLO:

Si usamos un cristal de cuarzo de 4 Mhz en el oscilador, podremos obtener un retardo igual a un microsegundo por cada instrucción NOP que insertemos en el código del programa:

RETARDO NOP

NOP

NOP

RETURN

Cada vez que llamemos a la subrutina RETARDO, obtendremos 3 microsegundos de demora.

RETFIE				RETFIE
Return from Interrupt				
Operación	TOS → PC 1 → GIE			
Sintaxis	[Etiqueta] RETFIE			
Operadores	No tiene			
Ciclos	2			
OPCODE	00	0000	0000	1001
Descripción	Carga el PC con el valor que se encuentra en la parte alta de la Pila, asegurando así la vuelta de la interrupción. Pone a 1 el bit GIE , con el fin de autorizar de nuevo que se tengan en cuenta las interrupciones.			

Registro de **STATUS**

PA2	PA1	PA0	TO#	PD#	Z	DC	C
-	-	-	-	-	-	-	-

EJEMPLO:

RETFIE

Al ejecutarse:

PC = TOS

GIE = 1

RETLW					RETLW
Retur with Literal in w					
Operación	$k \rightarrow w;$ $TOS \rightarrow PC$				
Sintaxis	[Etiqueta] RETLW k				
Operadores	$0 < k < 255$				
Ciclos	2				
OPCODE	11	01xx	kkkk	kkkk	
Descripción	Carga el registro w con el literal k , y después carga el PC con el valor que se encuentra en la parte superior de la PILA, efectuando así un retorno de subrutina.				

Registro de **STATUS**

PA2	PA1	PA0	TO#	PD#	Z	DC	C
-	-	-	-	-	-	-	-

EJEMPLO:

```

MOV LW 0x07 ;Se carga 07 h en w
CALL TABLA ;Tabla de valores
... ;w contiene en valor recogido
...
TABLA ADDWF PC ;Se añade a PC el desplazamiento (offset) de w
      RETLW k1 ;Nueva Tabla
      RETLW k2
      ...
      ...
      ...
      RETLW kn ;Fin de tabla

```

Al ejecutarse la instrucción w = toma el valor de k7

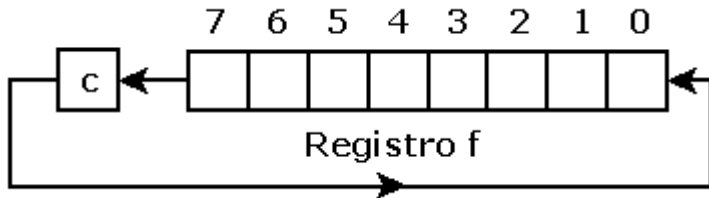
RETURN					RETURN
Return from Subroutine					
Operación	TOS → PC				
Sintaxis	[Etiqueta] RETURN				
Operadores	No tiene				
Ciclos	2				
OPCODE	00	0000	0000	0000	1000
Descripción	Carga el PC con el valor que se encuentra en la parte superior de la PILA, efectuando así un retorno de subrutina				

Registro de **STATUS**

PA2	PA1	PA0	TO#	PD#	Z	DC	C
-	-	-	-	-	-	-	-

EJEMPLO:

RETURN

RLF					RLF
Rotate Left f through Carry					
Operación					
Sintaxis	[Etiqueta] RLF f,d				
Operadores	$0 < f < 127$ d [0,1]				
Ciclos	1				
OPCODE	00	1101	dfff	ffff	
Descripción	Rotación de un bit a la izquierda del contenido del registro f, pasando por el bit de acarreo C, desde los bits menos significativos a los más significativos. El bit D7 pasa al CARRY del registro STATUS, el contenido del CARRY pasa al D0, el D0 al D1, etc. Es como si multiplicáramos por dos el contenido del registro. Si d=1 el resultado se almacena en f, si d=0 el resultado se almacena en w.				

Registro de STATUS

PA2	PA1	PA0	TO#	PD#	Z	DC	C
-	-	-	-	-	-	-	X

EJEMPLOS:

Si tenemos el registro VALOR = 0000 0001 b y aplicamos la instrucción **RLF VALOR,1**

Entonces el resultado será VALOR = 0000 0010 b y el bit C = 0.

Si tenemos el registro VALOR = 1110 0110 b y aplicamos la instrucción **RLF VALOR**

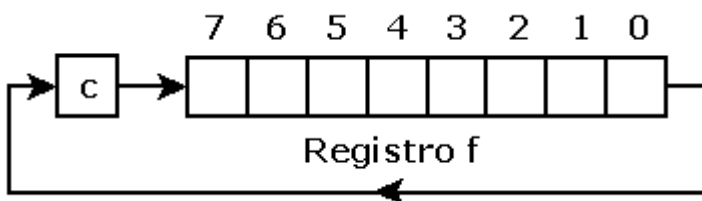
El resultado será VALOR = 1100 1100 b y el bit C = 1.

Si antes de la instrucción REG1 = 1110 0110 b y flag C = 0 y aplicamos la instrucción **RLF REG1,0**, como d = 0 el resultado queda en w, al ejecutarse:

REG1 = 1110 0110 b

w = 1100 1100 b

flag C = 1

RRF				RRF
Rotate Right f through Carry				
Operación				
Sintaxis	[Etiqueta] RRF f,d			
Operadores	0 < f < 127 d [0,1]			
Ciclos	1			
OPCODE	00	1100	dfff	ffff
Descripción	Rotación de un bit a la derecha del contenido del registro f , pasando por el bit de acarreo C , desde los bits más significativos a los menos significativos. El bit C del registro STATUS pasa al D7, el D0 pasa al bit C, el D1 al D0, etc. Es como si dividiéramos por dos el contenido del registro. Si d=1 el resultado se almacena en f , si d=0 el resultado se almacena en w			

Registro de **STATUS**

PA2	PA1	PA0	TO#	PD#	Z	DC	C
-	-	-	-	-	-	-	X

EJEMPLOS:

Si tenemos el registro VALOR = 0000 0001 b y aplicamos la instrucción **RRF VALOR,1**

Entonces el resultado será VALOR = 0000 0000 b y el bit C = 1.

Si tenemos el registro VALOR = 1000 0000 b y aplicamos la instrucción **RRF VALOR,1**

El resultado será VALOR = 0100 0000 b y el bit C = 0.

Si antes de la instrucción, REG1 = 1110 0110 b y flag C = 1 y aplicamos la instrucción **RRF REG1,0**, como d = 0 el resultado queda en w, al ejecutarse:

REG1 = 1110 0110 b

w = 0111 0011 b

flag C = 0

SLEEP					SLEEP
Sleep					
Operación	00 h → WDT 0 → WDT prescaler 1 → TO# 0 → PD#				
Sintaxis	[Etiqueta] SLEEP				
Operadores	No tiene				
Ciclos	1				
OPCODE	00	0000	0110	0011	
Descripción	Pone al circuito en modo Sleep (bajo consumo) con parada del oscilador. Pone a 0 el flag PD# (Power Down) y el flag TO# (Timer Out) se pone a 1. Se puede salir de este estado por: 1. Activación de MCLR para provocar un Reset. 2. Desbordamiento del watchdog si quedó operativo en el modo reposo. 3. Generación de una interrupción que no sea TMR0 ya que ésta se desactiva con la instrucción SLEEP.				

Registro de **STATUS**

PA2	PA1	PA0	TO#	PD#	Z#	DC	C
-	-	-	1	0	-	-	-

TO Se pone a 1 al ejecutar la instrucción SLEEP o CLRWDT

PD Se pone a 0 al ejecutar la instrucción SLEEP.

EJEMPLO:

SLEEP

SUBLW					SUBLW
Subtract w from Literal					
Operación	$k - w \rightarrow w$				
Sintaxis	[Etiqueta] SUBLW k				
Operadores	$0 < k < 255$				
Ciclos	1				
OPCODE	11	110x	kkkk	kkkk	
Descripción	Resta en complemento a dos del contenido del literal k el contenido del registro w , y almacena el resultado en w .				

Registro de **STATUS**

PA2	PA1	PA0	TO#	PD#	Z	DC	C
-----	-----	-----	-----	-----	---	----	---

-	-	-	-	-	X	X	X
---	---	---	---	---	---	---	---

Z Se pone a 1 si el resultado de la operación es cero

DC Se pone a 1 si se genera un acarreo del bit 3 al grupo de 4 bits superior

C Se pone a 1 si se genera un acarreo del bit de mayor peso.

EJEMPLO:

SUBLW 0x02 ;k - w → w, 02 h - w → w

a) Si antes de la instrucción w = 01 h y flag C = ? al ejecutarse:

02 h - 01 h = 01 h

w = 01 h

flag C = 1 ; el resultado es positivo

b) Si antes de la instrucción w = 02 h, flag C = ? y flag Z = ? al ejecutarse:

02 h - 02 h = 00 h

w = 00 h

flag C = 1

flag Z = 1 ; el resultado es cero

c) Si antes de la instrucción w = 03 h y flag C = ? al ejecutarse:

02 h - 03 h = -01 h = - 0000 0001 b

C1(0000 0001 b)=1111 1110 b; 1111 1110 b + 1 b = 1111 1111 b = FF h

w = FF h

flag C = 0 ; el resultado es negativo

SUBWF					SUBWF
Subtract w from f					
Operación	f - w → d				
Sintaxis	[Etiqueta] SUBWF f,d				
Operadores	0 < f < 127 d [0,1]				
Ciclos	1				
OPCODE	00	0010	dfff	ffff	
Descripción	Resta en complemento a dos el contenido del registro f menos el contenido del registro w almacena el resultado en w si d=0 y en f si d=1 .				

Registro de STATUS

PA2	PA1	PA0	TO#	PD#	Z	DC	C
-	-	-	-	-	X	X	X

Z Se pone a 1 si el resultado de la operación es cero

DC Se pone a 1 si se genera un acarreo del bit 3 al grupo de 4 bits superior

C Se pone a 1 si se genera un acarreo del bit de mayor peso.

EJEMPLO:

SUBWF REG1,1 ; $f - w \rightarrow f$, $REG1 - w \rightarrow REG1$

a) Si antes de la instrucción, REG1 = 03 h, w = 02 h y flag C = ?, al ejecutarse:

03 h - 02h = 01 h

REG1 = 01h

w = 02 h

flag C = 1 ; el resultado es positivo

b) Si antes de la instrucción, REG1 = 02 h, w = 02 h y flag C = ?, al ejecutarse:

02 h - 02h = 00 h

REG1 = 00h

w = 02 h

flag C = 1

flag Z = 1 ; el resultado es cero

c) Si antes de la instrucción, REG1 = 01 h, w = 02 h y flag C = ?, al ejecutarse:

01 h - 02 h = -01 h = - 0000 0001 b

C1(0000 0001 b)=1111 1110 b; 1111 1110 b + 1 b = 1111 1111 b = FF h

REG1 = FF h

w = 02 h

flag C = 0 ; el resultado es negativo

SWAPF				SWAPF
Swap Nibbles in f				
Operación	(f<3:0>) → (d <7:4>) (f<7:4>) → (d <3:0>)			
Sintaxis	[Etiqueta] SWAPF f,d			
Operadores	0 < f < 127 d [0,1]			
Ciclos	1			
OPCODE	00	1110	dfff	ffff
Descripción	Los cuatro bits de más peso del registro f se intercambian con los 4 bits de menos peso del mismo registro. Si d=0 el resultado se almacena en w , si d=1 el resultado se almacena en f .			

Registro de **STATUS**

PA2	PA1	PA0	TO#	PD#	Z	DC	C
-	-	-	-	-	-	-	-

EJEMPLO:

SWAPF REG1,0

Si antes de la instrucción:

REG1 = A5 h = 1010 0101 h

Como d=0 el resultado se almacenará en w

Al ejecutarse la instrucción:

REG1 = A5 h = 1010 0101 b

w = 5A h = 0101 1010 b

XORLW					XORLW
Exclusive OR Literal with k					
Operación	$\mathbf{w} \text{ XOR } \mathbf{k} \rightarrow \mathbf{w}$				
Sintaxis	[Etiqueta] XORLW <i>k</i>				
Operadores	$0 < \mathbf{f} < 255$				
Ciclos	1				
OPCODE	11	1010	kkkk	kkkk	
Descripción	Realiza la función OR-Exclusiva entre el contenido del registro w y la constante k de 8 bits. El resultado se almacena en w . Esta instrucción realiza la operación EXOR bit a bit.				

Registro de **STATUS**

PA2	PA1	PA0	TO#	PD#	Z	DC	C
-	-	-	-	-	X	-	-

Z Se pone a 1 si el resultado de la última operación es cero.

EJEMPLO:

XORLW 0xAF

Si antes de la instrucción:

w = 1011 0101 b = B5 h

Al ejecutarse la instrucción:

w = 1011 0101 b → 1010 1111 b = 0001 1010 b = 1A h

1011 0101 b

1010 1111 b

0001 1010 b

XORWF		XORWF
Exclusive OR w with f		
Operación	$w \text{ XOR } f \rightarrow d$	
Sintaxis	[Etiqueta] XORWF f,d	
Operadores	$0 < f < 127$ d [0,1]	
Ciclos	1	
Descripción	Realiza la función OR-Exclusiva entre el contenido del registro w y el contenido del registro f , y almacena el resultado en f si d=1 y en w si d=0 . Esta instrucción realiza la operación EXOR bit a bit.	

Registro de **STATUS**

PA2	PA1	PA0	TO#	PD#	Z	DC	C
-	-	-	-	-	X	-	-

EJEMPLO:

XORWF REG1,1

Si antes de la instrucción:

REG1 = AF h = 1010 1111 b

w = B5 h = 1011 0101 b

Como d=1, el resultado se almacena en REG1

Al ejecutarse:

REG1 = 1010 1111 $\rightarrow$ 1011 0101 = 0001 1010 = 1A h

w = B5 h

1010 1111

1011 0101

0001 1010

Instrucciones **OPTION** y **TRIS**

Entre las instrucciones anteriores no se han incluido dos que no pertenecen estrictamente hablando al repertorio de 35 instrucciones de la gama media. Estas instrucciones son **OPTION** y **TRIS**. La razón por la cual no pertenecen a estas 35 instrucciones es por que fueron creadas pensando en la gama baja, que carece de 4 de las instrucciones de la gama media: ADDLW, RETFIE, RETURN y SUBLW.

A pesar de todo las instrucciones **TRIS** y **OPTION** existen, en principio, en la gama media, pero Microchip recomienda no utilizarlas, para mantener la compatibilidad con todos los PIC de la gama media y los que puedan aparecer.

OPTION				OPTION
Guarda el valor del acumulador en el registro OPTION				
Operación	w → OPTION			
Sintaxis	[Etiqueta] OPTION			
Operadores	No tiene			
Ciclos	1			
OPCODE	00	0000	0110	0010
Descripción	Esta instrucción guarda en el registro especial OPTION el valor contenido en el acumulador w . No modifica ningún bit de estado.			

Registro de **STATUS**

PA2	PA1	PA0	TO#	PD#	Z	DC	C
-	-	-	-	-	-	-	-

EJEMPLO:

MOVLW 10H ; carga el acumulador con el valor 10h.

OPTION ; carga el registro OPTION con el acumulador.

Esta instrucción existe para mantener la compatibilidad con los PIC producidos con anterioridad, y como en el futuro podría dejar de implementarse, Microchip aconseja realizar el ejemplo anterior de esta otra forma:

BSF STATUS,RP0 ; activa el banco 1.

MOVLW 10H ; carga el acumulador con 10h

MOVWF OPTION_REG ; carga OPTION con el acumulador.

TRIS					TRIS
Guarda el acumulador en uno de los registros de TRIS .					
Operación	w → f (TRISA ó TRISB)				
Sintaxis	[Etiqueta] TRIS f				
Operadores	No tiene				
Ciclos	1				
OPCODE	00	0000	0110	1111	
Descripción	Esta instrucción guarda el valor del acumulador w en uno de los registros especiales TRIS (TRISA o TRISB) que indicamos en el parámetro f . No modifica ningún bit de estado. Los registros TRIS determinan el funcionamiento como entrada y salida de las líneas I/O del PIC.				

Registro de **STATUS**

PA2	PA1	PA0	TO#	PD#	Z	DC	C
-	-	-	-	-	-	-	-

EJEMPLO:

```
MOVLW 16h      ; carga el acumulador W con el valor 16h
TRIS    PORTA  ; carga el registro PORTA con el acumulador.
```

Esta instrucción existe para mantener la compatibilidad con los PIC producidos anteriormente, y como en el futuro podría dejar de implementarse, Microchip aconseja realizar el ejemplo anterior de esta otra forma (aunque ocupa más memoria...):

```
BSF     STATUS,RP0      ; activa el banco 1.
MOVLW 16h              ; carga el acumulador con el valor 16h
MOVWF  TRISA           ; carga el registro PORTA con W.
```

Instrucciones especiales

Existe un conjunto de instrucciones especiales diseñadas para facilitar las operaciones a la hora de diseñar nuestros algoritmos. Estas instrucciones pueden ser implementadas con una, dos o tres de las instrucciones de la gama media. La mayoría de ellas se basa en las operaciones con los acarreo y con los bits del registro status en general. Este cuadro sólo debe servir de referencia y se recomienda usar la forma equivalente del repertorio de instrucciones, no obstante, pueden encontrarse programas que los utilicen. Por supuesto con estos algoritmos, aunque utilicen una sola expresión, no vamos disminuir los ciclos máquina necesarios.

Mnemónico	Parámetros	Descripción	Traducción	Operación Equivalente	Flag
ADDCF	f, d	Add Carry to File	Sumar acarreo a f	BTFSC 3,0 INCF f,d	Z
ADDDCF	f, d	Add Digit Carry to File	Sumar acarreo de dígito a f	BTFSC 3,1 INCF f,d	Z
B	K	Branch	Saltar a una etiqueta	GOTO k	-
BC	K	Branch on Carry	Saltar a una etiqueta si hay acarreo	BTFSC 3,0 GOTO k	-
BDC	K	Branch on Digit Carry	Saltar a una etiqueta si hay acarreo de dígito	BTFSC 3,1 GOTO k	-
BNC	K	Branch on No Carry	Saltar a una etiqueta si no hay acarreo	BTFSS 3,0 GOTO k	-
BNDC	K	Branch on No Digit Carry	Saltar a una etiqueta si no hay acarreo de dígito	BTFSS 3,1 GOTO k	-
BNZ	K	Branch on No Zero	Saltar a una etiqueta si no hay cero	BTFSS 3,2 GOTO k	-
BZ	K	Branch on Zero	Saltar a una etiqueta si hay cero	BTFSC 3,2 GOTO k	-
CLRC		Clear Carry	Poner a cero acarreo	BCF 3,0	-
CLRDC		Clear Digit Carry	Poner a cero acarreo de dígito	BCF 3,1	-
CLRZ		Clear Zero	Poner a cero el flag Zero	BCF 3,2	-
LCALL	K	Long CALL	Llamada larga a una etiqueta	BSF/BCF 0A,3 BSF/BCF 0A,4 CALL k	-
LGOTO	K	Long GOTO	Salto largo a una etiqueta	BSF/BCF 0A,3 BSF/BCF 0A,4 GOTO k	-
MOVFW	F	Move File to W	Mover registro a W	MOVF f,0	Z
NEGF	f, d	Negate File	Negar un registro	COMF f,1 INCF f,d	Z
SETC		Set Carry	Poner a uno el acarreo	BSF 3,0	-
SETDC		Set Digit Carry	Poner a uno el acarreo de dígito	BSF 3,1	-
SETZ		Set Zero	Poner a uno el Zero	BSF 3,2	-
SKPC		Skip on Carry	Saltar si hay acarreo	BTFSS 3,0	-
SKPDC		Skip on Digit Carry	Saltar si hay acarreo de dígito	BTFSS 3,1	-
SKPNC		Skip on No Carry	Saltar si no hay acarreo	BTFSC 3,0	-
SKPNDC		Skip on No Digit Carry	Saltar si no hay acarreo de dígito	BTFSC 3,1	-
SKPNZ		Skip on Non Zero	Saltar si no hay Zero	BTFSC 3,2	-
SKPZ		Skip on Zero	Saltar si hay Zero	BTFSS 3,2	-
SUBCF	f,d	Subtract Carry from File	Restar acarreo del registro	BTFSC 3,0 DECf f,d	Z
SUBDCF	f,d	Subtract Digit Carry from File	Restar acarreo de dígito del registro	BTFSC 3,1 DECf f,d	Z

TSTF	f	Test File	Probar registro	MOVF f,1	Z
------	---	-----------	-----------------	----------	---
